

# ML final project

# Spotify Premium

Nàdia Pelegay Royo, Manon Vu Huu

# Contents

1. Introduction
2. Data
3. Comparative Analysis
4. kNN
5. kNN bis
6. Logistic regression
7. Random Forest Classifier
8. Making Playlist
9. (nadia)
10. Limitations & Discussion
11. Conclusion

# 1. Introduction



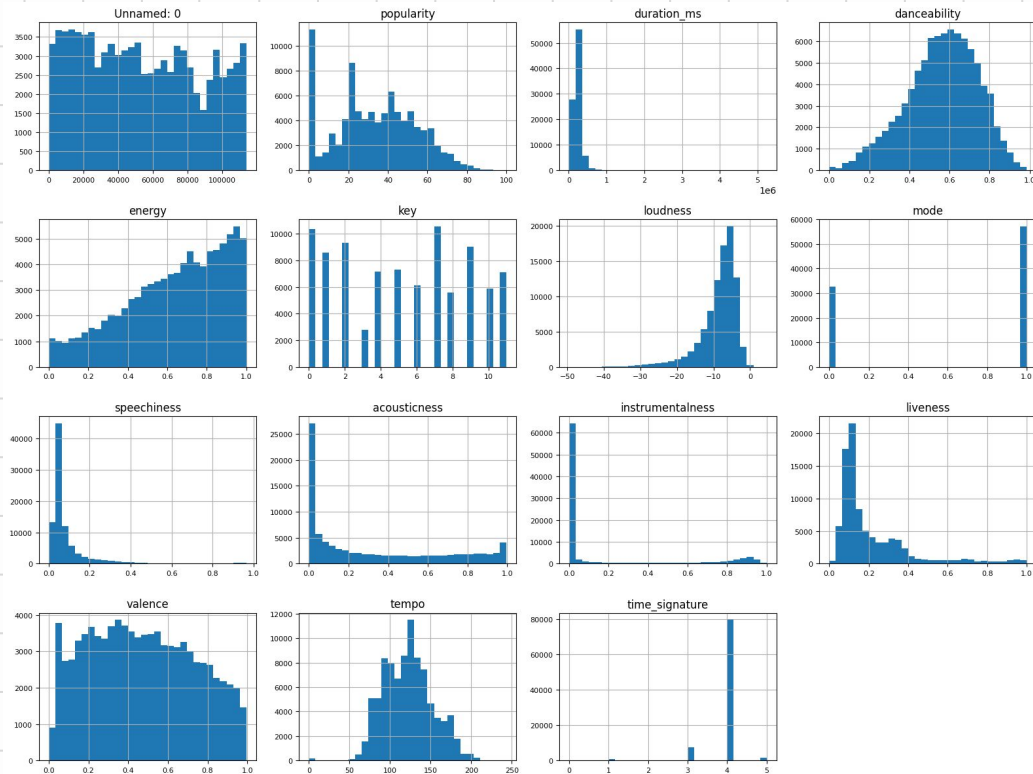
## 2. Data

```
import spotipy  
from spotipy.oauth2 import SpotifyOAuth, SpotifyClientCredentials  
import spotipy.util as util
```

```
path = kagglehub.dataset_download("maharshipandya/-spotify-tracks-dataset")
```

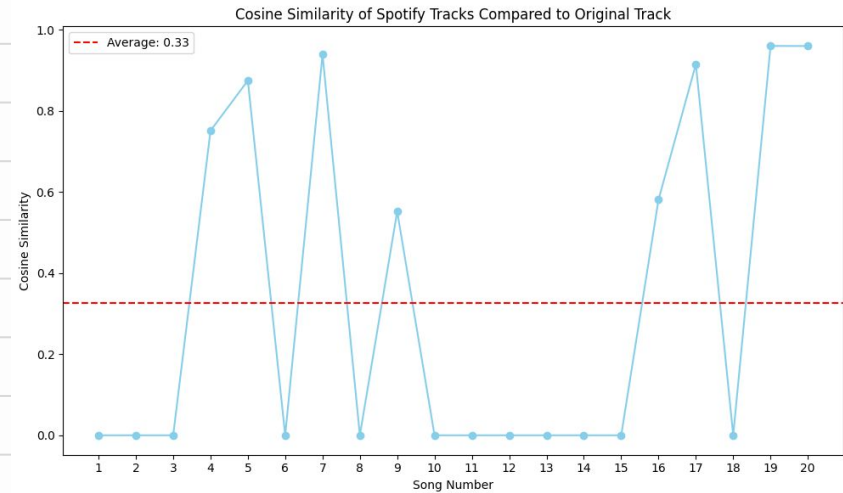
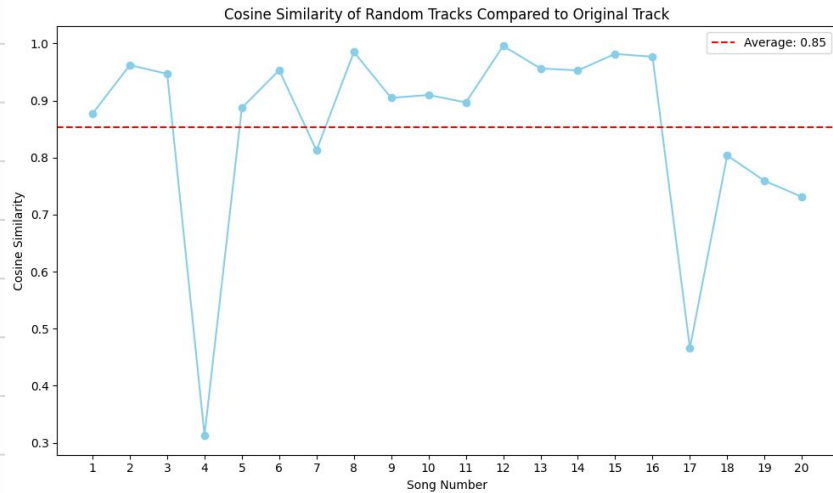
```
df = pd.read_csv(path+'/dataset.csv')
```

## 2. Data - Features



### 3. Comparative analysis - Cosine similarity

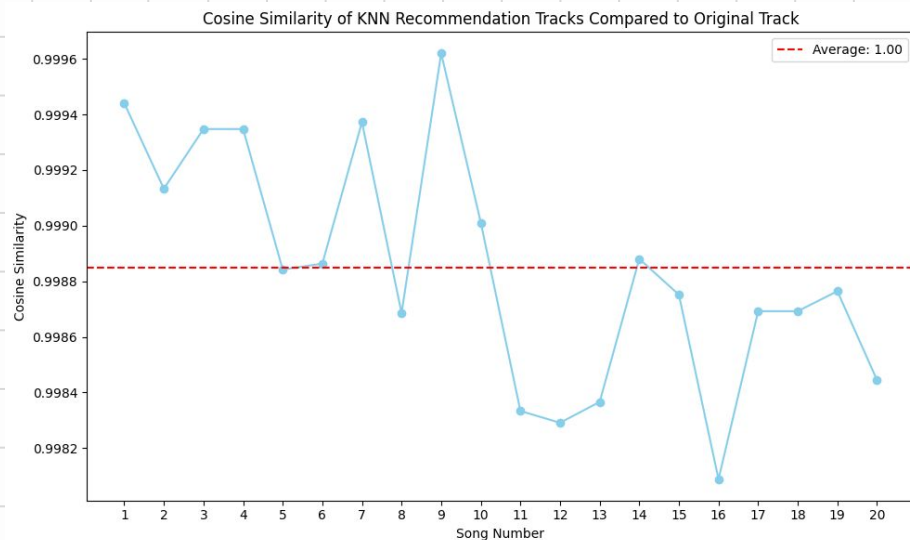
$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



## 4. kNN

```
knn_model = NearestNeighbors(n_neighbors=5)  
knn_model.fit(normalized_features)
```

```
def recommend_songs(track_id, nb=20):  
    song_index = df.index[df['track_id'] == track_id].tolist()[0]  
    distances, indices = knn_model.kneighbors([normalized_features[song_index]], n_neighbors=nb+1)  
    recommendations = df.iloc[indices[0][1:]]  
    return recommendations[['track_id', 'track_name', 'artists', 'album_name', 'track_genre']]
```



## 4. kNN - Interactive

```
Welcome to Nàdia and Manon's song recommender
Enter a track ID to base recommendations on: 57j0EZtoLQK4zF2x55bdkp
Do you want to apply any filters? (y/n): y
```

```
Available genres:
acoustic, afrobeat, alt-rock, alternative, ambient, anime, black-metal, bluegrass, b
Filter by genre (leave blank to skip): ambient
Minimum popularity (0-100, leave blank to skip): 20
Filter by artist (leave blank to skip):
```

```
Set feature emphasis (scale: 1 = neutral, >1 = prioritize, <1 = de-prioritize)
danceability (0-2, default 1): 1.4
energy (0-2, default 1): 1.3
valence (0-2, default 1): 1.2
tempo (0-2, default 1): 1.7
acousticness (0-2, default 1): 1
liveness (0-2, default 1): 1.4
speechiness (0-2, default 1): 0.6
instrumentalness (0-2, default 1): 0.9
Number of recommendations: 10
```

```
Recommended Songs:
```

# 5. kNN bis

```
def recommendation_flow(og_track_id, nb=10):
```

```
    visited = set([og_track_id])
    flow = []
```

```
    og_index = df.index[df['track_id'] == og_track_id].tolist()[0]
    now = normalized_features[og_index].reshape(1, -1)
```

```
    for _ in range(nb):
```

```
        #exclude already visited
```

```
        candidate_mask = df['track_id'].apply(lambda tid: tid not in visited)
```

```
        candidate_df = df[candidate_mask].reset_index(drop=True)
```

```
        candidate_vectors = MinMaxScaler().fit_transform(candidate_df[features])
```

```
        #knn
```

```
        knn = NearestNeighbors(n_neighbors=1)
```

```
        knn.fit(candidate_vectors)
```

```
        distance, index = knn.kneighbors(now)
```

```
        #get the next track
```

```
        next_row = candidate_df.iloc[index[0][0]]
```

```
        next_track_id = next_row['track_id']
```

```
        flow.append(next_row)
```

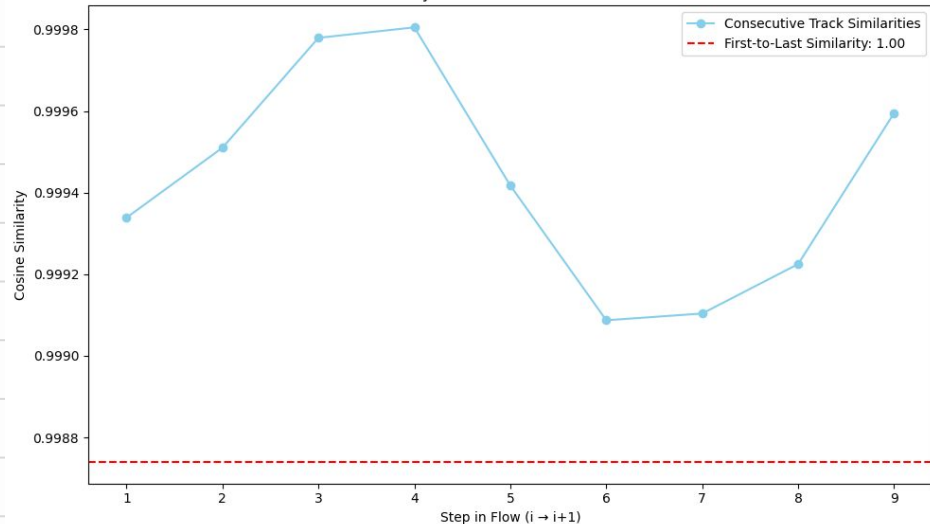
```
        visited.add(next_track_id)
```

```
        #update current vector
```

```
        now = candidate_vectors[index[0][0]].reshape(1, -1)
```

```
    return pd.DataFrame(flow)[['track_id', 'track_name', 'artists', 'album_name', 'track_genre']]
```

Cosine Similarity Between Consecutive Tracks in Flow



# 6. Logistic Regression

```
og = '2eAvDnpXP5W0cVtiI0PUxV'
threshold = 0.95

# Get index and vector for original song
og_index = df[df['track_id'] == og].index[0]
og_vector = normalized_features[og_index].reshape(1, -1)

# Compute cosine similarities for all tracks
all_similarities = cosine_similarity(normalized_features, og_vector).flatten()

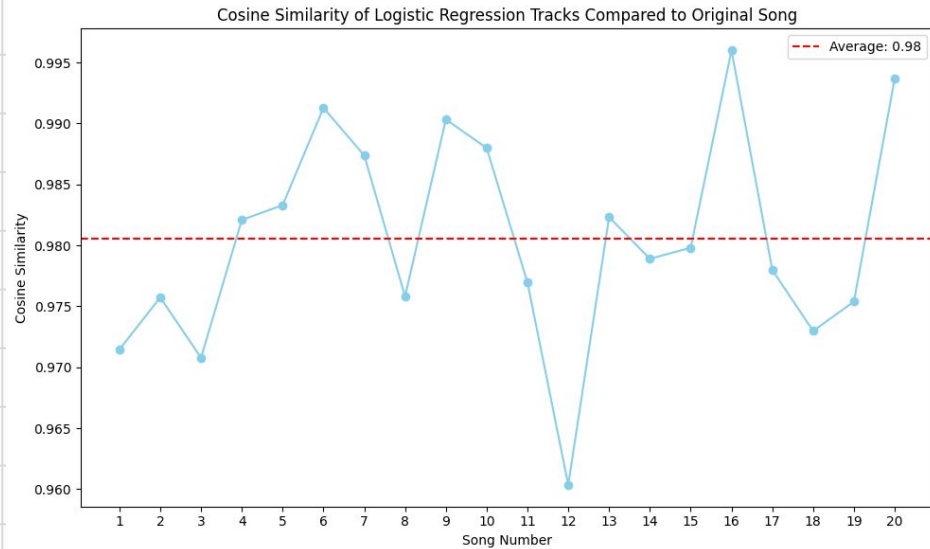
labels = (all_similarities >= threshold).astype(int)

# logistic regression
X = normalized_features
y = labels
clf = LogisticRegression()
clf.fit(X, y)

# probability of being similar
probabilities = clf.predict_proba(X)[: , 1]

# top n recommendations
n = 20
df['similarity_score'] = probabilities
recommendations = (
    df[df['track_id'] != og]
    .drop_duplicates(subset=['track_name', 'artists'])
    .sort_values(by='similarity_score', ascending=False)
    .head(n)
)

recommendations[['track_name', 'artists', 'similarity_score']]
```



# 7. Random Forest Classifier

```
og = '2eAvDnpXP5W0cVtiI0PUxV'
threshold = 0.95

#Get index and vector for original song
og_index = df[df['track_id'] == og].index[0]
og_vector = normalized_features[og_index].reshape(1, -1)

#Compute cosine similarities for all tracks
all_similarities = cosine_similarity(normalized_features, og_vector).flatten()

labels = (all_similarities >= threshold).astype(int)

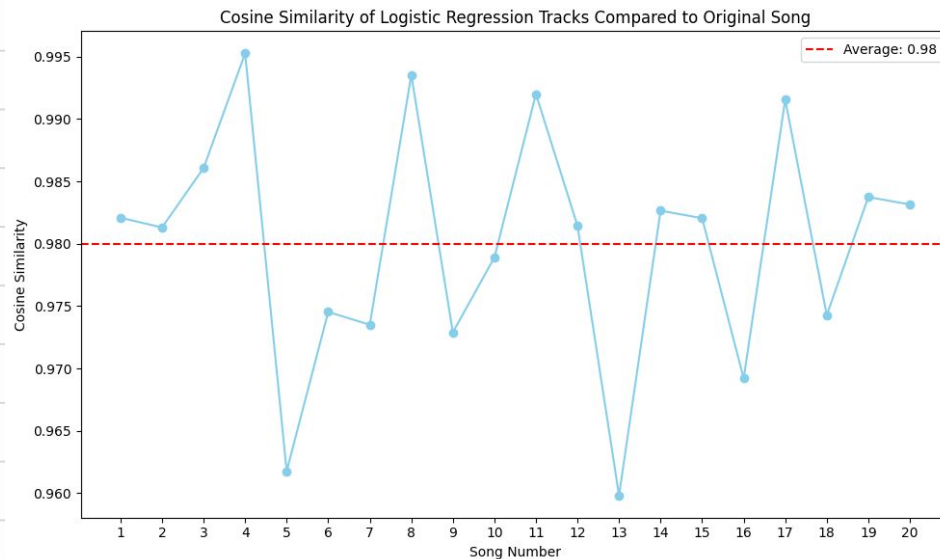
# Random Forest
X = normalized_features
y = labels
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X, y)

# probability of being similar
probabilities = clf.predict_proba(X)[: , 1]

# top n recommendations
n = 20
df['similarity_score'] = probabilities

recommendations = (
    df[df['track_id'] != og]
    .drop_duplicates(subset=['track_name', 'artists'])
    .sort_values(by='similarity_score', ascending=False)
    .head(n)
)

recommendations[['track_name', 'artists', 'similarity_score']]
```



## 8. Making Playlist

```
playlist_name = "dyi_recommendation" #to customize
playlist_description = "this was made by yours truly" #to customize
user_id = user['id']
recommendations = recommended[1] #one of the recommendation lists made above

new_playlist = sp.user_playlist_create(user=user_id, name=playlist_name, public=False, description=playlist_description)
playlist_id = new_playlist['id']
print(f"Created playlist '{playlist_name}' with ID: {playlist_id}")

sp.playlist_add_items(playlist_id=playlist_id, items=recommendations)
print(f"Added {len(recommendations)} tracks to playlist '{playlist_name}'")
```

## 9. Limitations & Discussions

- 3 year old dataset
- features
- Actual real life UX